

ACRME Calculation Logic — Plain English Walkthrough (v2.5)

Document type: Companion reference — plain-language explanation of every formula in the Calculation Logic Reference. This document does **not** replace the Calculation Logic Reference or the Technical Design Document. Read it alongside those documents to understand the intent behind each formula before reading the formal notation.

Audience: Platform engineers, product owners, and architects who need to reason about ACRME behaviour without working through mathematical notation first.

Version: 1.2 — reconciled to Requirements Baseline **v2.5** (24 September 2026).

v2.5 foundation note. Microsoft Learn corrections: sharing stays Preview and production placement uses a reservation in the deploying subscription; the consumer must hold its own quota (QUA-013, no longer an unknown); quota has a family cap and a regional total cap; a Quota Group transfers quota and the subscription is what deployment checks; a CRG is zonal by default and a no-zone group is pinned to one zone Azure chooses (CAP-023); logical zone numbers differ per subscription (CAP-016); deallocated associated VMs still hold quota until dissociated (CAP-004). Section 4.1 and Section 5.11.3 carry the detail.

v2.4 reconciliation note. The geography-aware region model has **three distribution models**: US three-region (Prod/CVAL/DR distinct); EU/Australia/Asia Pacific two-region co-located (CVAL+DR co-located per PLC-010a); and the **Middle East cross-geo DR model** — Prod+CVAL co-located in a weighted-selected Middle East region, DR placed cross-geo in a weighted-selected **Europe** Standard region (PLC-010b, DR-020, DEC-001 RESOLVED, A-ME1). The Middle East example in Section 3.6a reflects this. v2.4 added reservation **eligibility** (CAP-020/CAP-021), the **seed-at-0** SKU/AZ matrix (CAP-022), **reactive SKU/AZ discovery** (CAP-024), and the **even zone-distribution** target + rebalancing (PLC-011, Appendix A.9). The CRG shape in Section 5.11.3 is the v2.5 zonal default.

Document Control

Field	Value
Title	ACRME Calculation Logic — Plain English Walkthrough
Version	1.2 (reconciled to Requirements Baseline v2.5)
Status	Baseline
Date	24 September 2026 (foundation); originally 2 September 2026
Source	ACRME Q&A design walkthrough session; Calculation Logic Reference; Requirements Baseline v2.5
Companion documents	acrme_calculation_logic_reference.md (formal notation); ac-rme_technical_design_document.md (component design)

Table of Contents

Section	Title
1	Orientation — What ACRME Actually Does
2	How This Document Is Organised
3	Domain 1: Region Selection
4	Domain 2: Quota Management
5	Domain 3: Capacity Management
6	Scoring Weight Invariant
7	How the Three Domains Connect
8	Quick Reference — Key Numbers

1. Orientation — What ACRME Actually Does

Before any Azure VM is deployed for a customer, ACRME answers three questions in sequence:

1. **Where should this customer live?** — pick the best Azure region for each environment (production, CVAL, DR).
2. **Is there enough quota there?** — ensure the subscription has headroom to deploy the required VM family.
3. **Is there a physical reservation?** — ensure Azure has set aside the physical capacity the VM needs.

If all three answers are “yes,” the deployment gets a **READY** signal and proceeds. If any answer is “no,” the deployment is held until the gap is closed.

These three questions map to the three calculation domains this document explains:

Domain	Question answered	Core scenarios
Region Selection	Where does this customer go?	Scenarios 1, 2, 3, 4, 5, 6, 7
Quota Management	Is the subscription limit high enough?	Scenarios 8, 9, 10
Capacity Management	Has Azure reserved the physical hardware?	Scenarios 11, 12, 13, 16, 17, 18, 19

2. How This Document Is Organised

Each domain section follows the same pattern:

- **Concept** — what this domain manages, in plain language.
- **The formula, translated** — what each piece of the formula means.
- **Happy path** — what happens when everything is fine.
- **Sad path** — what happens when something is wrong.

Geography-specific examples use the authoritative Standard Capacity region sets (Requirements Baseline

v2.4 Section 6, ADR-001). **Five geographies are in scope. US is the only three-region geography; all**

others use a two-region distribution model — Prod in one region, CVAL + DR co-located in the other

(PLC-010a). **[Amended v2.4]** The **Middle East is the exception**: it uses a **cross-geo DR model** — Prod+CVAL co-locate in a weighted-selected Middle East region and DR is placed cross-geo in a weighted-selected Europe region (PLC-010b, DR-020). The catalogue is a configurable item (REG-001), so this table is the current example, not a hard-coded set:

Geography	Distribution model	Standard Capacity Regions (engine-selectable)	Restricted Regions (exception only)
US	3-region	West US 3 · Central US · Canada Central	East US 2
EU	2-region	Switzerland North · Sweden Central	North Europe · West Europe
Australia	2-region	Australia East · Australia Southeast	—
Asia Pacific	2-region	East Asia · Southeast Asia	Japan East (pending confirmation)
Middle East	2-region	UAE North · Saudi Arabia Central	—

The two-region walkthrough below uses **EU** as the worked example; **Australia and Asia Pacific follow the exact same 2-region co-located pattern** (Prod in one region, CVAL + DR co-located in the

other). **[Amended v2.4] The Middle East is different:** Prod and CVAL co-locate in a weighted-selected

Middle East region, but **DR is placed cross-geo in a weighted-selected Europe Standard region** (PLC-010b, DR-020, DEC-001 RESOLVED) — see Section 3.6a.

Representative SKU examples throughout: **E16ads_v5** (16 vCPU per VM) and **E8ads_v5** (8 vCPU per VM).

3. Domain 1 — Region Selection

3.1 Concept

Region selection answers the question: “Of all the Azure regions available for this geography, which one should this customer’s production (or CVAL, or DR) environment live in?”

The engine scores every eligible region and picks the highest-scoring one. The score is a number between 0 and 1 — higher is better. The region with the highest score wins.

Selection happens in a fixed sequential order: **Prod first, then CVAL (NonProd), then DR.**

- The **Prod region** is usually supplied by the customer. ACRME validates it against hard constraints rather than deriving it. Deriving a Prod region from geography alone is an exception path that requires explicit approval.
- The **CVAL region** is selected by the engine from the remaining Standard regions after Prod is fixed.
- The **DR region** is selected last from the remaining Standard regions.

If a customer's supplied region is a **Restricted Capacity Region** (e.g. East US 2, North Europe), it does not enter scoring at all. It goes to a manual exception workflow.

3.2 The Hard Constraint Gate — Ruling Out Ineligible Regions

Before any scoring happens, each candidate region must pass a set of hard constraints (HC-1..HC-10). Think of these as eligibility checks that must all pass before a region is even considered. A region that fails any hard constraint is removed from the scoring candidate pool entirely.

The three arithmetic constraints that matter most in practice:

HC-3 (Quota Floor): Does the region have enough quota to accept the new deployment?

For Prod: the region's Prod quota headroom $\geq$ the vCPUs needed for this deployment
 For NonProd: the region's effective NonProd headroom $\geq$ the vCPUs needed
 For DR: the region's DR headroom $\geq$ the target DR quantity $\times$ vCPU

In plain English: the region's subscription quota limit is high enough that adding these VMs will not push usage over the cap.

HC-6 (DR Coverage Floor): Does the region have enough room in its DR and NonProd CRGs combined to take on this customer's DR requirement?

HC-7 (DR Floor Integrity): Would adding these NonProd VMs eat into the space that is earmarked for DR? If yes, the region is rejected — ACRME never lets NonProd growth erode DR protection.

What happens if a region fails? It is excluded from the scoring step. The engine scores only the regions that passed all hard constraints.

3.3 The Scoring Formulas — What Each Weight Means

Each of the three environment types has its own scoring formula. All three share the same five weights

($\alpha=0.30$, $\beta=0.20$, $\gamma=0.25$, $\delta=0.15$, $\epsilon=0.10$) but each weight measures something slightly different depending on the environment type.

Every component value is **clamped to the range [0, 1]** before being multiplied by its weight. This means a component can never go below 0 or above 1, regardless of the raw numbers.

Why These Specific Weights? The Building Analogy

Before diving into the technical formulas, here is the plain-English intuition behind each weight, using the analogy of placing desks in office buildings:

Weight	Value	Plain English reason (building/office analogy)
$\alpha = 0.30$ (largest)	Headroom/capacity signal	The single most important factor. If a building is nearly full, we cannot grow — and growing is inevitable. Deserves the highest priority.
$\gamma = 0.25$	Distribution fairness	We never want all customers in the same building. If one building catches fire (fails), we want as few customers affected as possible. Second highest.
$\beta = 0.20$	Quota headroom	The building permit (quota) must have room approved by the authority (Microsoft). Without it, no new desks can be added even if the floor is empty.
$\delta = 0.15$	DR readiness	Is the backup office already covering enough customers? A well-covered DR region is a better destination. Lower priority because DR coverage depends on the customer mix, not just space.
$\epsilon = 0.10$ (smallest)	Zone diversity	Are the desks spread across different floors of the building? If one floor has a power cut, other floors keep running. Nice-to-have, not critical.

All five add to exactly 1.0 (100%). They are **policy defaults stored in config** — not hardcoded. The team can adjust them if priorities change (see Section 6 for the weight-sum validation requirement).

Alternative Configuration: Distribution-First Priority

The weights above reflect the **baseline default** where capacity headroom is the top priority. However,

if **minimizing blast radius** (spreading customers across regions) becomes more important than keeping

maximum headroom, the weights can be adjusted. Here are two alternative configurations:

Option 1: Distribution-First (Conservative)

Weight	Baseline	Alternative	What changes
γ (Distribution)	0.25	0.35	Now highest — spreading customers is the top goal
α (Capacity headroom)	0.30	0.25	Reduced — still important but no longer dominant
β (Quota headroom)	0.20	0.20	Unchanged — quota is always a hard gate
δ (DR readiness)	0.15	0.12	Slightly reduced
ϵ (Zone diversity)	0.10	0.08	Slightly reduced
Sum	1.0	1.0	✓ Valid

Building analogy for distribution-first: “We never want all customers in the same building. If one building catches fire (fails), we want as few customers affected as possible. **This is now our highest priority** — we’re willing to run buildings a bit fuller if it means better customer spread.”

Option 2: Distribution-First (Aggressive)

Weight	Baseline	Alternative	What changes
γ (Distribution)	0.25	0.40	Heavily dominant — blast radius minimization is paramount
α (Capacity headroom)	0.30	0.20	Significantly reduced — willing to accept tighter capacity
β (Quota headroom)	0.20	0.20	Unchanged
δ (DR readiness)	0.15	0.12	Reduced
ϵ (Zone diversity)	0.10	0.08	Reduced
Sum	1.0	1.0	✓ Valid

Trade-offs of distribution-first weighting:

- ✓ **Better blast radius control** — regional failures affect fewer customers
- ✓ **More even spread** across regions — reduces hot spots
- ✓ **Better alignment with reliability/resilience goals**
- ⚠ **May place workloads in regions with less headroom** → faster capacity exhaustion

-  **Engine will score a crowded-but-balanced region higher** than an empty-but-unbalanced one
-  **Could trigger more frequent quota/capacity increases** if chosen regions fill faster

See Section 3.8 for the full Weight Tuning Guide.

Component α (weight 0.30) — Headroom or Capacity Signal

This is the single biggest factor. It answers: “How much free capacity does this region have?”

Formula type	What it measures
PS_Prod	NonProd CRG effective free slots ÷ Prod CRG quantity (how much NonProd breathing room backs the Prod anchor)
PS_NonProd	NonProd CRG effective free slots ÷ NonProd CRG quantity (direct NonProd headroom ratio)
PS_DR	DR CRG free slots ÷ DR CRG quantity (direct DR headroom ratio)

In plain English: a region with more available headroom scores higher. This is the most heavily weighted factor because placing a customer in a region with no headroom creates an immediate risk of capacity exhaustion.

Component β (weight 0.20) — Quota Headroom Signal

Answers: “How much room does the subscription’s quota limit have?”

All three environment types measure this the same way: the ratio of remaining quota headroom to total

quota limit. A region where the subscription is nearly at its quota limit scores lower. This matters because even if Azure has physical capacity, a quota limit prevents deployment.

Component γ (weight 0.25) — Distribution Fairness

Answers: “Is this region already heavily loaded with customers?”

All three environment types measure this as $1 - (\text{customers already in this region} \div \text{total customers across all regions})$.

The subtraction from 1 means a lightly loaded region scores high (close to 1) and a heavily loaded region scores low (close to 0).

In plain English: ACRME tries to spread customers evenly. A region that already hosts most of the customer base is less attractive for the next customer — even if it has adequate headroom.

Component δ (weight 0.15) — DR Readiness or Overflow Integrity Signal

Answers: “Is this region’s DR position healthy?”

Formula type	What it measures
PS_Prod	DR CRG coverage ratio (does this region have adequate DR coverage already?)
PS_NonProd	NonProd CRG effective free ÷ quantity (overflow capacity health — duplicate of α in design-of-record)
PS_DR	DR CRG coverage ratio ÷ dr_ratio_target (how close is DR coverage to the bootstrap target?)

In plain English: this component rewards regions where the DR position is healthy — the region is not already overcommitted from a DR perspective.

Component ϵ (weight 0.10) — Zone Diversity

Answers: “How many Availability Zones does this region have?”

The formula is $\text{az_count} \div 3$, clamped to $[0, 1]$. A region with 3 AZs scores 1.0; a region with 1 AZ scores 0.33. Zone diversity matters because it determines whether the deployment can be spread across multiple physical failure domains.

In plain English: a region with more Availability Zones is more resilient and scores higher on this component, all else equal.

Important: The Weights Are NOT Calculated — The Components Are

A common point of confusion: α , β , γ , δ , ϵ themselves are never calculated. They are **fixed policy constants** ($\alpha=0.30$, $\beta=0.20$, $\gamma=0.25$, $\delta=0.15$, $\epsilon=0.10$) stored in `PlacementPolicy` and loaded at runtime. They only change if an operator edits the policy file (see Section 3.8).

What the engine **calculates at runtime**, per region, is the **component value** that each weight multiplies — the `$\alpha$ _component`, `$\beta$ _component`, etc. Each component is a ratio derived from the **regional snapshot** (the live, versioned state of each region’s reservations, quota, customers, and zones). The formula is always:

```
weight (fixed policy constant) × component (calculated from the snapshot)
```

So “how is α calculated?” really means “how is the **α _component** calculated from the snapshot?” — which is what the next two subsections show with concrete numbers.

Regional Snapshot — Assumed Starter Values

The engine reads a **regional snapshot** before scoring. A snapshot is the current state of every candidate region. The raw fields below are the inputs to the component formulas. The values here are **assumed starter values** for a worked example (US Geography, `PS_Prod` scoring, deploying a Prod workload). In production these come from live Azure state + the reservation store; here they are fixed so the arithmetic is reproducible.

Snapshot field	West US 3	Central US	Canada Central	Used by
prod_crg.quantity (reserved Prod slots)	200	200	200	α (denominator)
non-prod_crg.effective_free (free NonProd slots after DR overflow reserve)	130	80	40	α (numerator)
prod_crg.quota_limit (vCPU)	1000	1000	1000	β (denominator)
prod_crg.quota_headroom (vCPU free below limit)	700	550	300	β (numerator)
prod_customer_count (customers already in region)	40	25	20	γ
total_customers (across all regions) ⚠	100	100	100	γ
dr_crg.coverage_ratio (DR coverage already in place)	0.50	0.45	0.60	δ
az_count (Availability Zones)	3	3	2	ϵ

⚠ **SPEC GAP (GAP- γ):** The `total_customers` term in the γ component is **undefined** in Requirements Baseline v2.5. Its source (customer registry? subscription count? tenant-level aggregate?), scope (all customers or only those with Prod workloads?), and data type are not specified. The γ formula $1 - \text{prod_customer_count} / \text{total_customers}$ cannot be implemented without resolving this gap. See the **OPEN ISSUE** section in `acrme_calculation_logic_comprehensive_walkthrough.md` for four potential interpretations and a proposed resolution action plan. Values shown here (100) are illustrative placeholders only.

Note on “starter” values. These are illustrative seed values for the walkthrough, not baseline defaults. The only true baseline defaults are the weights (Section 3.3) and the configurable parameters (Section 8.1). Snapshot values are always live data, never configuration.

Worked Calculation — Deriving Each Component (West US 3)

Using the `PS_Prod` formulas from Section 3.3 and the snapshot row for **West US 3**. Every component is clamped to $[0, 1]$ via `Clamp(x) = max(0, min(1, x))`.

```

α_component = Clamp(nonprod_crg.effective_free / prod_crg.quantity)
               = Clamp(130 / 200) = Clamp(0.65) = 0.65

β_component = Clamp(prod_crg.quota_headroom / prod_crg.quota_limit)
               = Clamp(700 / 1000) = Clamp(0.70) = 0.70

γ_component = Clamp(1 - prod_customer_count / total_customers)
               = Clamp(1 - 40 / 100) = Clamp(0.60) = 0.60

δ_component = Clamp(dr_crg.coverage_ratio)
               = Clamp(0.50) = 0.50

ε_component = Clamp(az_count / 3)
               = Clamp(3 / 3) = Clamp(1.0) = 1.0

```

Now apply the fixed weights:

```

PS_Prod(West US 3) = 0.30 × 0.65    (α)
                   + 0.20 × 0.70    (β)
                   + 0.25 × 0.60    (γ)
                   + 0.15 × 0.50    (δ)
                   + 0.10 × 1.00    (ε)
                   = 0.195 + 0.140 + 0.150 + 0.075 + 0.100
                   = 0.66

```

Repeating the same derivation for the other two regions from the snapshot:

Component (calculated)	West US 3	Central US	Canada Central
$\alpha_{\text{component}} = \text{effective_free} / \text{prod quantity}$	$130/200 = \mathbf{0.65}$	$80/200 = \mathbf{0.40}$	$40/200 = \mathbf{0.20}$
$\beta_{\text{component}} = \text{quota_headroom} / \text{quota_limit}$	$700/1000 = \mathbf{0.70}$	$550/1000 = \mathbf{0.55}$	$300/1000 = \mathbf{0.30}$
$\gamma_{\text{component}} = 1 - \text{customers} / \text{total}$	$1 - 0.40 = \mathbf{0.60}$	$1 - 0.25 = \mathbf{0.75}$	$1 - 0.20 = \mathbf{0.80}$
$\delta_{\text{component}} = \text{dr coverage_ratio}$	$\mathbf{0.50}$	$\mathbf{0.45}$	$\mathbf{0.60}$
$\epsilon_{\text{component}} = \text{az_count} / 3$	$3/3 = \mathbf{1.0}$	$3/3 = \mathbf{1.0}$	$2/3 = \mathbf{0.67}$
PS_Prod (weighted sum)	0.66	0.58	0.48

These are exactly the component values used in the Section 3.5 US walkthrough — that section

applies these same numbers to pick `argmax(PS_Prod) = West US 3`. The snapshot above is where those numbers come from.

Key takeaways:

- The **weights** (0.30, 0.20, 0.25, 0.15, 0.10) are identical for every region — they are policy.
- The **components** differ per region — they are calculated from that region's snapshot.
- A change in live state (e.g., West US 3 fills up, dropping `effective_free`) changes only the components, never the weights, and can flip which region wins.

3.4 Putting It Together — The Score and the Winner

Each region gets a score between 0 and 1:

```
Score(region) = 0.30 ×  $\alpha_{\text{component}}$ 
               + 0.20 ×  $\beta_{\text{component}}$ 
               + 0.25 ×  $\gamma_{\text{component}}$ 
               + 0.15 ×  $\delta_{\text{component}}$ 
               + 0.10 ×  $\epsilon_{\text{component}}$ 
```

The region with the **highest score wins** — this is what the formula `argmax(PS)` means. If two regions tie, the engine picks the first one in the Standard Capacity Region list for that geography (deterministic tie-break).

The winning region's score, along with every input value and the PlacementPolicy version, is written to the OperationRecord so the decision can be replayed or audited later.

3.5 US Geography Walkthrough (3 Standard Regions)

Context: US Geography has three Standard Capacity regions: West US 3, Central US, Canada Central. All three are initially in the scoring candidate pool.

Happy Path — All Three Regions Eligible

All three regions pass HC-1..HC-10. The engine scores all three and picks the highest:

Candidate regions: { West US 3, Central US, Canada Central }

Score(West US 3): $0.30 \times 0.65 + 0.20 \times 0.70 + 0.25 \times 0.60 + 0.15 \times 0.50 + 0.10 \times 1.0 = 0.66$

Score(Central US): $0.30 \times 0.40 + 0.20 \times 0.55 + 0.25 \times 0.75 + 0.15 \times 0.45 + 0.10 \times 1.0 = 0.58$

Score(Canada Central): $0.30 \times 0.20 + 0.20 \times 0.30 + 0.25 \times 0.80 + 0.15 \times 0.60 + 0.10 \times 0.67 = 0.48$

argmax → West US 3 (Prod anchor)

Subsequent CVAL and DR selections run the same scoring against the remaining eligible regions.

Sad Path — One Region Fails the Hard Constraint Gate

Suppose Central US has a current NonProd quota usage so high that adding 3× E16ads_v5 (48 vCPU) would exceed the effective NonProd ceiling:

HC-7 check for Central US:

$\text{nonprod_quota_used} + (3 \times 16) = \text{nonprod_quota_used} + 48 > \text{effective_nonprod_ceiling}$
→ HC-7 FAIL → Central US excluded from scoring pool

The engine now scores only { West US 3, Canada Central }. It still picks the best of the remaining two — it does not fail the entire placement. Only if **all** Standard regions fail the hard constraint gate does the engine fall into the exception workflow.

3.6 EU Geography Walkthrough (2 Standard Regions)

Context: EU Geography has only two Standard Capacity regions: Switzerland North and Sweden Central.

This is a structural constraint: with only two Standard regions, the engine cannot place Prod, CVAL, and DR in three separate regions.

How EU handles this — the co-location rule (PLC-010 / PLC-010a): one region is Prod, the other is

both CVAL and DR together (co-located). CVAL and DR share the second region. This is by design, not a

fallback. In a two-region geography the co-location is **mandatory and deterministic** — once Prod is anchored, only one region remains, so CVAL and DR must land there (PLC-010a).

The `CVALEarmarkRecord` prevents double-counting: CVAL capacity earmarked for DR activation cannot simultaneously be counted as live NonProd headroom.

Note: DR_NOT_OFFERED does not apply to EU. Both Switzerland North and Sweden Central are fully eligible for all three environment types.

Happy Path — Both Regions Eligible

Candidate regions: { Switzerland North, Sweden Central }

Step 1 (Prod): $\text{argmax}(\text{PS_Prod}) \rightarrow$ say Switzerland North scores highest $\rightarrow$ Prod = Switzerland North

Step 2 (CVAL): only Sweden Central remains $\rightarrow$ CVAL = Sweden Central (deterministic, no argmax needed)

Step 3 (DR): same region as CVAL by PLC-010 co-location rule $\rightarrow$ DR = Sweden Central

CustomerSeedRecord:

production_region = "Switzerland North"

cval_region = "Sweden Central"

dr_region = "Sweden Central" $\leftarrow$ co-located with CVAL

Sad Path — One Region Fails the Hard Constraint Gate

If Sweden Central fails HC-3 (insufficient Prod quota headroom for this deployment):

HC-3 check for Sweden Central:

prod_quota_headroom < requested_vm_count $\times$ vCPU

$\rightarrow$ HC-3 FAIL $\rightarrow$ Sweden Central excluded from Prod scoring

Only remaining Prod candidate: Switzerland North

Prod = Switzerland North (deterministic – no argmax , single candidate)

CVAL + DR must go to Sweden Central (only remaining Standard region) regardless of HC failure for Prod

scoring. The engine re-evaluates HC for CVAL/NonProd context – a region that fails Prod constraints

may still pass NonProd constraints because they check different quota groups/pools.

If **both** regions fail the gate for the same environment type, the engine has no eligible candidate and escalates to the exception workflow (Scenario 3).

3.6a Australia, Asia Pacific & Middle East — Same 2-Region Pattern

EU is not special. **Every two-region geography behaves identically** (PLC-010a): the placement logic

depends only on the count of Standard regions in the geography (a configurable value, REG-001), not on which geography it is. Substitute the region names and the walkthrough above applies verbatim.

Australia (Standard: Australia East, Australia Southeast):

Candidate regions: { Australia East, Australia Southeast }

Step 1 (Prod): $\text{argmax}(\text{PS_Prod}) \rightarrow$ say Australia East scores highest $\rightarrow$ Prod = Australia East

Step 2 (CVAL): only Australia Southeast remains $\rightarrow$ CVAL = Australia Southeast (deterministic)

Step 3 (DR): co-located with CVAL by PLC-010a $\rightarrow$ DR = Australia Southeast

CustomerSeedRecord:

production_region = "Australia East"

cval_region = "Australia Southeast"

dr_region = "Australia Southeast" $\leftarrow$ co-located with CVAL

Asia Pacific (Standard: East Asia, Southeast Asia; Japan East pending confirmation):

```
Candidate regions: { East Asia, Southeast Asia } ← Japan East excluded until confirmed in PlacementPolicy

Step 1 (Prod):  argmax(PS_Prod) → say Southeast Asia scores highest → Prod = Southeast Asia
Step 2 (CVAL):  only East Asia remains → CVAL = East Asia (deterministic)
Step 3 (DR):    co-located with CVAL by PLC-010a → DR = East Asia

CustomerSeedRecord:
  production_region = "Southeast Asia"
  cval_region       = "East Asia"
  dr_region         = "East Asia" ← co-located with CVAL
```

Middle East (Standard: UAE North, Saudi Arabia Central) — **[Amended v2.4] the cross-geo DR exception:** Prod and CVAL co-locate in a weighted-selected Middle East region (separate CRGs), and **DR is placed cross-geo in a weighted-selected Europe Standard region** (PLC-010b, DR-020, DEC-001 RESOLVED). Both selections use the weighted capacity model:

```
Prod/CVAL candidate regions (Middle East): { UAE North, Saudi Arabia Central }
DR candidate regions (Europe Standard):    { Switzerland North, Sweden Central, ... }
← weighted-selected

Step 1 (Prod):  weighted argmax(PS_Prod) over ME → say UAE North scores highest → Prod = UAE North
Step 2 (CVAL):  co-located with Prod in ME → CVAL = UAE North (separate CRG; co-location ≠ sharing)
Step 3 (DR):    weighted selection over Europe Standard regions → say Switzerland North → DR = Switzerland North (cross-geo)

CustomerSeedRecord:
  production_region = "UAE North"
  cval_region       = "UAE North" ← co-located with Prod locally (separate CRG)
  dr_region         = "Switzerland North" ← cross-geo, weighted-selected Europe region (example)
```

The Europe DR destination sizes DR **max-not-sum** (DR-017) across all sources that fail over to it, including Middle East sources. The specific Europe region is weighted-selected (Switzerland North is the default example, not fixed); changing the Europe destination set is a config change, no code change (REG-001, REG-002, HC-10). Because CVAL now co-locates with Prod locally (not with DR), the CVAL-sacrifice DR bootstrap (DR-005/006) does not apply to the Middle East — ME DR uses dedicated reserved capacity in Europe. Per-country data-residency carve-outs are config-driven and out of engine scope (A-ME1).

3.7 Worked Example — E16ads_v5 (16 vCPU) at US Scale

Deployment: 3× E16ads_v5 = 48 vCPU for Prod in US Geography.

HC-3 Prod check for each candidate region:

Required: 48 vCPU

Region must have Prod_Group_headroom $\geq$ 48 vCPU

If West US 3 has Prod_Group_headroom = 64 vCPU $\rightarrow$ PASS ($64 \geq 48$)

If Central US has Prod_Group_headroom = 32 vCPU $\rightarrow$ FAIL ($32 < 48$) $\rightarrow$ excluded

Scoring runs on: { West US 3, Canada Central }

Winner: argmax of those two

3.8 Weight Tuning Guide — How to Adjust Priorities

This section explains **how** and **when** to adjust the five scoring weights (α , β , γ , δ , ϵ) to reflect different business priorities, and what happens when you do.

3.8.1 The Weight-Sum Constraint

CRITICAL RULE: The five weights must **always sum to exactly 1.0**. This is a mathematical requirement enforced by the Config/Scope-File Service at policy load time (see Section 6).

$$\alpha + \beta + \gamma + \delta + \epsilon = 1.0$$

Why this matters:

- When weights sum to 1.0, placement scores range from 0.0 (worst) to 1.0 (best)
- Scores are directly comparable across all regions
- Each weight represents its **exact contribution** to the final score
- Validation tolerance: ± 0.001 (to handle IEEE 754 floating-point rounding)

If weights don't sum to 1.0, the policy is rejected and a `PolicyValidationError` is raised.

3.8.2 What Each Weight Controls

Weight	What it measures	When to increase	When to decrease
α	Capacity headroom	Growth is frequent; deployments are large; running out of capacity is costly	Utilization is more important than headroom; willing to run regions fuller
β	Quota headroom	Microsoft quota increases are slow; hitting quota limits blocks deployments	Quota is abundant; rarely hit limits
γ	Distribution fairness	Blast radius minimization is critical; want even customer spread	Hot spots are acceptable; concentration is not a concern
δ	DR readiness	DR coverage is a top-tier SLA/compliance requirement	DR is lower priority; willing to accept gaps
ϵ	Zone diversity	Zone-level failures are common; zone spread is critical	Single-zone is acceptable; zones are equally reliable

3.8.3 Common Tuning Scenarios

Scenario A: Headroom-First (Baseline Default)

“Growth is inevitable and unpredictable. Never run out of capacity.”

Weight	Value	Rationale
α	0.30	Highest — capacity headroom is paramount
γ	0.25	Second — still want distribution
β	0.20	Third — quota is a hard gate
δ	0.15	Fourth — DR is important but secondary
ϵ	0.10	Lowest — zones are least critical

Use this when: Your workload grows rapidly, deployments are large, and running out of capacity would block business-critical launches.

Scenario B: Distribution-First (Blast Radius Control)

“Minimize blast radius. Spread customers evenly even if it means tighter capacity.”

Weight	Value	Rationale
γ	0.35	Highest — customer spread is paramount
α	0.25	Second — still need some headroom
β	0.20	Unchanged — quota is always a gate
δ	0.12	Reduced — DR is secondary
ϵ	0.08	Lowest — zones are least critical

Use this when: Reliability/resilience is the top goal, and you want to minimize the number of customers affected by a single regional failure — even if it means placing workloads in fuller regions.

Scenario C: DR-Readiness-First (Compliance-Driven)

“DR coverage is a contractual/compliance SLA. DR must always be ready.”

Weight	Value	Rationale
δ	0.30	Highest — DR readiness is the top goal
α	0.25	Second — still need capacity
γ	0.20	Third — distribution matters
β	0.15	Fourth — quota is secondary
ϵ	0.10	Lowest — zones are least critical

Use this when: You have contractual/regulatory DR commitments (e.g., SOC 2, financial services), and DR readiness must be continuously validated and reported.

Scenario D: Quota-Constrained Environment

“Microsoft quota increases are slow. Never hit quota limits.”

Weight	Value	Rationale
β	0.30	Highest — quota headroom is the bottleneck
α	0.25	Second — capacity matters but quota blocks first
γ	0.20	Third — distribution is secondary
δ	0.12	Reduced — DR is lower priority
ϵ	0.13	Higher than baseline — zones matter more here

Use this when: You operate in regions where Microsoft quota increases take weeks, or you have frequent quota-limit blocks. Prioritize regions with abundant quota headroom.

Scenario E: Balanced / All-Equal (Testing/Development)

“Treat all factors equally for testing or when priorities are unclear.”

Weight	Value	Rationale
α	0.20	Equal weight
β	0.20	Equal weight
γ	0.20	Equal weight
δ	0.20	Equal weight
ϵ	0.20	Equal weight

Use this when: You’re testing the engine, experimenting with scoring, or priorities are genuinely equal (rare in production).

3.8.4 The Pie-Rule: Every Increase Requires a Decrease

Because weights must sum to 1.0, **every increase to one weight requires a decrease to one or more**

others. Think of it as a fixed-size pie: making one slice bigger forces others to shrink.

Example: Increasing γ (distribution) from 0.25 to 0.40

You need to **remove 0.15** from other weights:

```

γ: 0.25 → 0.40 (+0.15)
α: 0.30 → 0.20 (-0.10)
δ: 0.15 → 0.12 (-0.03)
ε: 0.10 → 0.08 (-0.02)
β: 0.20 → 0.20 (unchanged)

```

Sum: 1.0 → 1.0 ✓

Questions to ask when tuning:

1. Which factor is **most critical** to your business right now?
2. Which factor can you **afford to de-prioritize**?
3. What are the **trade-offs** of the change?
4. Can you **test** the new weights in a non-production scope first?

3.8.5 How to Change Weights

Per the requirements baseline (C-2, ENV-005) and Technical Design Document (Section 8.2):

1. **Update the PlacementPolicy configuration file** (not code)
 - Weights are stored as tunable policy constants
 - Changes apply per scope hierarchy: product/region/environment/SKU
 - Defaults can be overridden per scope
2. **Config/Scope-File Service validates the weight-sum at policy load time**
 - Validation gate: `|sum - 1.0| < 0.001`
 - If valid → policy is published
 - If invalid → `PolicyValidationError` is raised; deployment blocked
3. **Every placement decision records the policy_version used**
 - Enables deterministic replay
 - Provides audit trail of configuration state at decision time
4. **Test matrix T-WV-01..T-WV-05** validates the weight-sum gate
 - See Calculation Logic Reference (Section on Policy weight-sum validation rule)

No code deployment is required to change weights. Only a policy configuration update.

3.8.6 Monitoring the Impact of Weight Changes

After changing weights, monitor these metrics to validate the impact:

Metric	What to watch	Expected change
Regional capacity utilization	Average % full across regions	Higher if α decreased; lower if α increased
Customer concentration	Max customers in any single region	Lower if γ increased; higher if γ decreased
Quota increase requests	Frequency of quota requests to Microsoft	Higher if β decreased; lower if β increased
DR coverage ratio	% of production covered by DR capacity	Higher if δ increased; lower if δ decreased
Zone spread	% of deployments spanning 3 zones	Higher if ϵ increased; lower if ϵ decreased
Placement score distribution	Histogram of region scores	Shape changes based on which weights dominate

Best practice: Change weights **incrementally** (e.g., ± 0.05 at a time) and monitor for 1–2 weeks before making further adjustments.

3.8.7 Per-Environment Weight Customization

Weights can be tuned **per environment type** if Prod, NonProd, and DR have different priorities:

Example: Prod prioritizes headroom, NonProd prioritizes distribution

```
Prod weights:
   $\alpha=0.35$ ,  $\beta=0.20$ ,  $\gamma=0.20$ ,  $\delta=0.15$ ,  $\epsilon=0.10$  (headroom-first)

NonProd weights:
   $\gamma=0.35$ ,  $\alpha=0.25$ ,  $\beta=0.20$ ,  $\delta=0.12$ ,  $\epsilon=0.08$  (distribution-first)

DR weights:
   $\delta=0.30$ ,  $\alpha=0.25$ ,  $\gamma=0.20$ ,  $\beta=0.15$ ,  $\epsilon=0.10$  (DR-readiness-first)
```

This allows **different placement strategies per environment** while maintaining a single engine.

3.8.8 Summary: Weight Tuning Checklist

Before changing weights, verify:

- ☒ **Sum equals 1.0** (± 0.001 tolerance)
- ☒ **Business rationale documented** — why this change, what problem it solves
- ☒ **Trade-offs understood** — what you're gaining and what you're giving up
- ☒ **Test scope identified** — which product/region/environment will get the new weights first
- ☒ **Monitoring plan in place** — how you'll measure impact
- ☒ **Rollback plan defined** — how to revert if the change causes issues
- ☒ **Policy version recorded** — audit trail for compliance

The five weights are the engine’s “personality.” Tuning them changes **how** the engine makes decisions, but the decision **process** (hard constraints → scoring → ranking) remains the same.

4. Domain 2 — Quota Management

4.1 Concept

Quota is the Azure-imposed limit on how many vCPUs a subscription can deploy in a region. There are **two** limits, and a deployment needs room under both:

- the **VM-family** vCPU quota (for example `Eadsv5`)
- the **total regional** vCPU quota for that subscription in that region

Even if physical capacity exists, deployment fails if either limit is exhausted. Creating a reservation consumes the same two quotas.

ACRME accounts for family quota as **one cell per region per VM family**, covering Prod, NonProd/CVAL,

and DR together. Inside the cell, ACRME uses **logical earmarks** to protect Prod and DR space — they are never available for NonProd to consume.

That cell is bookkeeping inside **one Azure Quota Group per subscription**. The group can span regions and families, and each transfer is for one region and one family. A VM create does not read the group limit. It reads the quota sitting on the deploying subscription after any transfer. A family that looks free while the regional total is exhausted is still a quota deficit.

4.2 The Single Pool — Why One Pool?

The “one pool” below is the **accounting cell** (region × VM family, all environments). It is not a second Azure object, and it is not the number Azure checks at deploy time. Moving quota onto the deploying subscription is still an explicit Quota Group transfer (QUA-003).

Older designs used separate quota groups (one for Prod, one for NonProd+DR). The single-cell model was adopted because:

- **Flexibility:** all quota is in one place. If Prod needs more room, there is no inter-group transfer required — the engine just allocates from the shared pool.
- **Efficiency:** quota is expensive to obtain from Microsoft (quota increase requests take time and require justification). Pooling avoids the situation where one group is near-empty while another has idle headroom.
- **Emergency DR draw:** during an active DR event, the DR orchestrator draws directly from `Pool_Headroom`. No cross-group transfer; no Azure quota request needed for Tier 2 operations.

The two-group topology is retained as a narrow exception when Azure’s own Quota Group API limits or a mandatory governance boundary make one pool impossible.

4.3 The Pool Formula — Translated

```
Pool_Limit(region) =
  Prod CRG quantity × vCPU × 1.20    (Prod with 20% growth buffer)
+ NonProd CRG quantity × vCPU × 1.20 (NonProd with 20% growth buffer)
+ DR_Earmark_vCPU(region)             (space permanently reserved for DR standby)
+ emergency_transfer_headroom_vcpu    (30% of potential DR demand – Tier 2 buffer)
```

What this means in plain English: the pool limit is sized to be big enough to hold everything at full scale, including a growth allowance, a permanent DR reserve, and a buffer for emergency DR capacity draws. When ACRME requests this quota from Microsoft, it is asking for the headroom to cover all of these uses without coming back for more quota frequently.

4.4 Logical Earmarks — Protecting Prod and DR Inside the Pool

Once the pool exists, the engine enforces two internal floors that NonProd can never consume:

Prod_Reserved_Floor: The minimum quota the Prod workloads need — their current vCPU usage plus the growth buffer above it. NonProd allocation is blocked if it would eat below this floor.

DR_Earmark_vCPU: The quota reserved for DR standby. This is calculated using the max-not-sum formula (see Domain 3 — Scenario 17): it is the capacity needed to absorb the largest single source region that could fail and send its customers here, not the sum of all potential sources.

Allocatable_NonProd is what is left over after both earmarks:

```
Allocatable_NonProd = Pool_Limit
                    - Prod_Reserved_Floor
                    - DR_Earmark_vCPU
                    - NonProd currently used
```

If this number hits zero, the engine blocks any further NonProd expansion in that region until either the pool limit is raised or Prod/DR demand decreases.

4.5 The DR Floor — Why It Exists and How It Is Enforced

The DR floor (`DR_Earmark_vCPU`) is a ring-fence: the engine promises that there will always be enough quota to activate DR for the largest protected source region. Without it, NonProd growth could quietly eat the DR quota until a real DR event arrives — at which point the DR activation fails.

Azure itself does not natively enforce intra-pool sub-reservations. The engine enforces the DR floor by arithmetic at command time (when NonProd expansion is requested) and by a separate detector that

continuously recomputes the floor from authoritative assignment data. If the detector disagrees with the command-time calculation, automatic NonProd expansion is disabled until the state is reconciled.

4.6 US Geography Walkthrough — Quota Sizing

Context: 3× E16ads_v5 (48 vCPU) for Prod, 3× E8ads_v5 (24 vCPU) for NonProd, in US Geography (3 Standard regions: West US 3, Central US, Canada Central). DR requirement sized by max-not-sum.

Worked example for one region (West US 3):

Inputs:

```

Prod CRG quantity      = 48 vCPU  (3× E16ads_v5 × 16 vCPU)
NonProd CRG quantity   = 24 vCPU  (3× E8ads_v5  × 8 vCPU)
DR_Earmark_vCPU        = 48 vCPU  (sized for the largest source – see Domain 3)
emergency_transfer_headroom = 24 × 0.30 = 7.2 → ceil → 8 vCPU

```

```

Pool_Limit = (48 × 1.20) + (24 × 1.20) + 48 + 8
            = 57.6 + 28.8 + 48 + 8
            = 142.4 → request 143 vCPU from Microsoft

```

```

Prod_Reserved_Floor    = 48 + (48 × 0.20) = 57.6 vCPU
DR_Earmark_vCPU         = 48 vCPU
NonProd currently used = 24 vCPU (all in use)

```

```

Allocatable_NonProd = 143 - 57.6 - 48 - 24 = 13.4 vCPU remaining

```

Happy Path — NonProd Expansion Request Within Allocatable Headroom

A team requests 1 additional E8ads_v5 (8 vCPU) for NonProd:

```

HC-7 check: NonProd used + 8 = 24 + 8 = 32 > Allocatable_NonProd ceiling?
Allocatable_NonProd ceiling = Pool_Limit - Prod_Reserved_Floor - DR_Earmark_vCPU
                           = 143 - 57.6 - 48 = 37.4 vCPU

32 ≤ 37.4 → PASS
NonProd expansion approved.

```

Sad Path — NonProd Expansion Would Violate DR Floor

A team requests 4× E8ads_v5 (32 vCPU) for NonProd at once:

```

HC-7 check: NonProd used + 32 = 24 + 32 = 56 > effective NonProd ceiling of 37.4 vCPU
→ HC-7 FAIL
→ Emit: DRFloorViolationDetected (Critical severity)
→ Block expansion
→ Alert: "NonProd expansion denied – DR floor integrity violated"

```

The team must request a quota increase from Microsoft (raising Pool_Limit) before this deployment can proceed.

4.7 EU Geography Walkthrough — Quota with Co-location

In EU, CVAL and DR co-locate in the second Standard region (say Sweden Central). The pool for Sweden

Central must cover both NonProd and DR earmarks:

```

Prod region:      Switzerland North
CVAL + DR region: Sweden Central

```

Sweden Central pool covers:

```

NonProd CRG quantity = 24 vCPU
DR_Earmark_vCPU       = sized for DR requirement
(Prod_Reserved_Floor applies only in Switzerland North)

```

```

CVALEarmarkRecord enforces: CVAL capacity used for DR activation is NOT
simultaneously counted as Allocatable_NonProd headroom.

```

This prevents a double-count: CVAL VMs that are earmarked to be shut down and replaced by DR standby cannot be claimed as “available NonProd headroom” while they are still running.

4.8 Quota Scoring (Scenario 10) — The β Component

The β component of the placement score (weight 0.20) reflects the current quota position. It is dispatched by environment type:

```
Quota_Score_Prod(r)    = prod quota headroom ÷ prod quota limit
Quota_Score_NonProd(r) = NonProd headroom ÷ effective NonProd ceiling
Quota_Score_DR(r)     = DR headroom ÷ DR floor vCPU
```

Important nuance on DR: a DR score of 1.0 means maximum expansion room (DR CRG is at zero — nothing is reserved yet, so the ceiling has not been eaten). It does not mean “quota is fully consumed.” A DR score of 0 means the DR earmark is fully used — the region cannot take more DR load.

5. Domain 3 — Capacity Management

5.1 Concept

Quota (Domain 2) answers whether the subscription limit allows a deployment. Capacity management (Domain 3) answers whether Azure has actually set aside physical hardware for those VMs.

This is managed through **Capacity Reservation Groups (CRGs)**. A CRG tells Azure: “I want you to guarantee that a certain number of VMs of this SKU in this region/zone will always be available to me.” Azure marks that physical hardware as reserved; other customers cannot take it.

ACRME’s capacity management job is to keep the CRG quantity correctly sized — not too high (wastes money) and not too low (exposes customers to deployment failures).

5.2 The Reconciliation Floor — The Minimum the CRG Must Reserve

The most fundamental formula is:

```
Target Reserved Capacity = Allocated VM Count + Configured Buffer
```

Allocated VM Count is the number of VMs currently in `running` or `allocated` state consuming compute in this region/CRG.

Configured Buffer is the policy-defined headroom above current demand. It is tunable per product/region/environment/SKU — there is no single hard-coded number.

What this means in plain English: at minimum, the CRG must reserve enough physical capacity to cover every running VM plus a buffer for expected near-term growth. The buffer is there because acquiring new reservation capacity takes time — ACRME stays ahead of demand.

Important clarification — deallocated VMs (CAP-004): VMs that are associated with a CRG but currently deallocated (stopped/not running) do **not** raise the capacity target. Target stays Allocated + Buffer. They are reported separately so teams understand restart risk.

They **do** still consume the quota charged to the reservation. Azure counts them in `virtualMachinesAssociated` until they are dissociated. Reconciliation does not treat that quota as free, and it does not shrink the reserved quantity below the associated count while they remain associated.

5.3 Reservation Headroom and Deficit

```
Reservation Headroom = Reserved Quantity - Allocated VM Count
Reservation Deficit   = max(0, Target Reserved Capacity - Reserved Quantity)
```

Headroom tells the operator how much spare capacity is immediately available inside the existing reservation. **Deficit** tells the operator how far short the reservation is of the minimum target. A deficit of zero means the CRG is at or above the floor. A positive deficit means the CRG is undersized and needs to grow.

5.4 Auto-Increase Thresholds — Configurable Trigger Points

Before explaining when ACRME raises capacity requests, it's important to understand that the trigger points are **configurable policy values**, not hardcoded constants.

5.4.1 The Three Auto-Increase Threshold Parameters

ACRME uses three configurable thresholds stored in `PlacementPolicy`:

Parameter Name	Default Value	What it measures
<code>prod_autoincrease_threshold</code>	0.20 (20%)	Remaining headroom % — triggers when Prod capacity has only 20% free slots left (80% utilized)
<code>non-prod_autoincrease_threshold</code>	0.20 (20%)	Remaining headroom % — triggers when NonProd capacity has only 20% free slots left (80% utilized)
<code>dr_autoincrease_threshold</code>	0.35 (35%)	Remaining headroom % — triggers when DR capacity has only 35% free slots left (65% utilized)

Key point: These represent **remaining headroom**, not consumption. A threshold of 0.20 means “trigger when only 20% headroom remains,” which is the same as “trigger when 80% is consumed.”

Why they're configurable:

- **Workload-specific risk tolerance** — high-growth products may want a 0.30 threshold for earlier warning
- **Cost vs risk trade-off** — lower thresholds (e.g., 0.10) run fuller but increase capacity exhaustion risk
- **Per-environment tuning** — DR can tolerate tighter thresholds because it's a bootstrap model
- **Regional variation** — regions with slow Azure provisioning times may warrant higher thresholds

Configurability scope: Like all PlacementPolicy parameters, these can be overridden per product/region/environment/SKU without code changes.

5.4.2 How Auto-Increase Thresholds Relate to Reconciliation Logic

The capacity reconciliation engine operates on a **continuous watch loop**:

```
Every reconciliation_interval_seconds (configurable):
  1. Read current state (allocated VMs, reserved capacity)
  2. Calculate headroom: free_slots = reserved - allocated
  3. Calculate headroom ratio: headroom_ratio = free_slots / reserved
  4. Check threshold: if headroom_ratio < threshold, trigger auto-increase
  5. Apply debounce: if triggered, wait 30 min cooldown before next trigger
```

Example: Prod CRG with 100 reserved slots

```
Allocated VMs: 78
Free slots: 100 - 78 = 22
Headroom ratio: 22 / 100 = 0.22 (22%)

Threshold check:
  prod_autoincrease_threshold = 0.20
  0.22 > 0.20 → No trigger (still 2% above threshold)

Next cycle, allocated grows to 82:
  Free slots: 100 - 82 = 18
  Headroom ratio: 18 / 100 = 0.18 (18%)
  0.18 < 0.20 → TRIGGER auto-increase request

Debounce: Next trigger for this (region + Prod CRG) cannot fire for 30 min
```

Interaction with the reconciliation floor (Section 5.2):

- The **reconciliation floor** (`Allocated + Buffer`) is the **minimum** the CRG must always reserve
- The **auto-increase threshold** is the **early warning** that triggers proactive expansion
- Both work together: the floor prevents deficits, the threshold prevents getting close to deficits

Why DR has a higher threshold (0.35 vs 0.20):

DR reservations are intentionally sized lean under the minimal-bootstrap model (ENV-005). A higher threshold (35% remaining = 65% utilized) allows DR to run fuller before triggering an increase. This trades slightly higher risk for lower steady-state cost, which is acceptable because:

- DR capacity is **standby/inactive** until a declared event
- During a DR event, staged acquisition (DR-006) allocates available capacity + CVAL sacrifice + pooled quota
- The 35% headroom is sufficient for control-plane bootstrap and initial recovery orchestration

Tuning guidance:

- **Increase threshold (e.g., 0.20 → 0.30)** if you want **earlier warnings** and more safety margin
- **Decrease threshold (e.g., 0.20 → 0.15)** if you want to **run fuller** and tolerate tighter headroom
- **Monitor** the frequency of auto-increase triggers — too frequent = threshold too low; never triggering = threshold too high

See Section 8 for the complete Configurable Parameters Reference.

5.5 The Auto-Increase Trigger — When ACRME Raises a Capacity Request

ACRME does not wait for a deficit to appear before acting. It watches utilisation in real time and triggers a capacity increase request when utilisation hits the configured thresholds (Section 5.4.1):

Environment	Default threshold	Meaning
Prod	20% headroom remaining	Trigger when only 20% of reserved Prod capacity is free (80% utilized)
NonProd	20% headroom remaining	Trigger when only 20% of reserved NonProd capacity is free (80% utilized)
DR	35% headroom remaining	Trigger when only 35% of reserved DR capacity is free (65% utilized)

Debounce cooldown: after any trigger fires, the engine waits 30 minutes before it can fire again for the same region + CRG type. This prevents repeated requests from a transient spike.

Why the DR threshold is higher (35% vs 20%): DR reservations are intentionally sized lean — the min-bootstrap model means DR starts small and scales on demand. A higher threshold accepts more risk in exchange for lower cost during steady state.

5.6 The Forecast Formula — Proactive Growth

Alongside the continuous reconciliation floor, ACRME uses an advisory forecast for longer-horizon sizing:

```
Forecast_Quantity = ceil( Forecast_Peak × (1 + Growth_Buffer) + DR_Buffer )
```

- **Forecast_Peak** is the predicted peak VM demand over the chosen horizon (30, 60, or 90 days).
- **Growth_Buffer** is a policy percentage for forecast uncertainty.
- **DR_Buffer** is the extra units required by DR recovery policy.

This is advisory — recommendations from the forecast are reviewed before acting. The horizon and buffer values are stored in PlacementPolicy, not hard-coded.

Lead-time alerting: when forecast demand is projected to reach 80% of the quota limit within 14 days, the engine emits `ForecastApproachingQuotaLimit`. This gives the platform team 14 days to request a quota increase from Microsoft before the deployment pipeline hits a wall.

5.7 DR Sizing — Why Max, Not Sum (Scenario 17)

This is the biggest cost-driver decision in ACRME. Before v2.1, DR standby was sized as a **percentage of production capacity** (30–40%). This was found to over-provision by \$1.5M–\$5M/year at platform scale.

The replacement formula is:

```
Destination_DR_Requirement(destination_region) =
    MAX over all source regions that send DR customers to this destination (
        the quantity of VMs/vCPUs from that source that must failover here
    )
```

Why MAX and not SUM? The design assumes **only one production region fails at a time** (the single-failure assumption, DR-001). Because failures are mutually exclusive, the destination never needs to host simultaneous failovers from multiple sources. It only ever absorbs the largest one.

Example from Requirements Appendix D:

```
Sweden Central hosts DR standby for:
→ Switzerland North customers: 120 cores
→ (hypothetically) a third EU region: 80 cores

SUM sizing (old): 120 + 80 = 200 cores required
MAX sizing (new): max(120, 80) = 120 cores required
Saving: 80 cores (40%) removed with no loss of protection
```

If the single-failure assumption is ever violated (two regions fail simultaneously), the standby is overcommitted. The Overcommit_Ratio quantifies this exposure and is required input for the POC-011 risk sign-off that gates production reliance on max-not-sum.

A **SUM override (C-11)** is available per-scope in PlacementPolicy for customers or geographies that contractually require protection against concurrent regional failures.

5.8 Happy Path — Steady-State Capacity Management

All conditions normal for West US 3, Prod CRG, E16ads_v5:

```
Allocated VM Count      = 3  (the 3 production VMs are running)
Configured Buffer        = 2  (policy: always keep 2 extra slots ready)
Target Reserved         = 5

Reserved Quantity       = 5
Reservation Headroom    = 5 - 3 = 2
Reservation Deficit     = max(0, 5 - 5) = 0

Utilisation = 3 / 5 = 60%
20% threshold not breached (utilisation is above threshold in the wrong direction –
this
    would actually be concerning, but the threshold fires when utilisation hits the
threshold
    from below, meaning the CRG is filling up)
```

Wait — clarifying the threshold direction: the auto-increase trigger fires when utilisation of the CRG **exceeds** the threshold, i.e., the CRG is getting full. At 60% utilisation on a Prod CRG (threshold 20%), the CRG is already well above threshold — the engine would have already raised a capacity increase request earlier, when utilisation first hit 20%.

Revised happy path:

```

Allocated VM Count      = 1 of 5 reserved slots used (20% utilisation)
Utilisation = 1 / 5 = 20%
Threshold    = 20% → trigger fires → CapacityIncreaseRequest raised

```

The request is raised, debounce cooldown starts (30 min), operator approval obtained (Phase 1), new CRG quantity updated.

5.9 Sad Path — Azure Cannot Supply the Requested Capacity

After ACRME raises a capacity increase request and submits it to Azure, Azure responds that it cannot supply the requested SKU/zone at this time:

```

Azure response: CAPACITY_UNAVAILABLE for E16ads_v5 in West US 3, Zone 2
ACRME action:
  1. Hold current CRG quantity (do not reduce below current level).
  2. Raise alert: "Capacity unavailable – reservation increase failed for West US 3 / E16ads_v5 / Zone 2".
  3. Expose buffer deficit in the observability dashboard.
  4. Do NOT delete or reduce the reservation object (CAP-009/010).
  5. Retry according to the reconciliation cadence (target 6-minute cycle).
  6. Escalate to emergency transfer tiers if the deficit is critical and a DR event is active.

```

The deployment readiness gate (Scenario 19) returns `CAPACITY_UNAVAILABLE` for any new deployment request that depends on this CRG until the situation resolves.

5.10 Standby Activation During DR (Scenario 18)

When a DR event is declared and the engine enters `DR_EVENT_ACTIVE` :

1. Standby VMs (currently `associated` with the destination CRG but not running) transition to `allocated` (active, consuming capacity).
2. This happens in **priority waves**: Wave 0 (platform control planes first), then Wave 1 (highest-priority customers), then subsequent waves in business-priority order.
3. Each wave goes through up to 7 stages of capacity acquisition:
 - Stages 1-2: use pre-staged bootstrap headroom and existing reservation — zero Azure wait time.
 - Stage 3: shut down eligible CVAL workloads (after authorisation) to free their slots.
 - Stages 4-6: share reservations, draw pooled quota, request additional Azure capacity.
 - Stage 7: report unrecoverable gaps.

The design ensures P0/P-1 customers begin recovery without waiting on Azure deallocation APIs — which are throttled during regional failure events.

5.11 v2.4 Reservation-Model Additions (Plain English)

Requirements Baseline v2.4 folded the reservation-model detail from the reviewed architecture diagrams (Reservation Model, Reservation Creation, Reservation Decommissioning) into the baseline. Nothing above changes; the following concepts are added.

5.11.1 Which VMs can even have a reservation? (CAP-020, CAP-021)

Not every VM is eligible for a capacity reservation:

- **Availability-Set VMs are ineligible (CAP-020).** A VM deployed into an Azure **Availability Set** cannot be backed by a capacity reservation. Capacity reservations are an **availability-zone** concept; Availability Sets are the older, non-zonal placement model, and Azure does not let the two mix. In plain terms: if a workload is in an Availability Set, ACRME will not try to reserve capacity for it — it flags it as ineligible with the reason recorded.
- **Onboarding a running VM needs a deallocate-or-redeploy step first (CAP-021).** To bring an existing running VM under the reservation model, it must first be placed into an availability zone. That means either **deallocating** it and restarting it into a zone, or **redeploying** it to a zone. ACRME treats this as an onboarding **precondition**: until the VM is zone-aligned, it cannot join the zonal reservation for that zone. The engine records this precondition state so operators know exactly what step is outstanding.

Both facts are captured in a `ReservationEligibility` record (the reason it is or isn't eligible, and the onboarding precondition status).

5.11.2 Reservations start at zero — the seed matrix (CAP-022)

ACRME does **not** guess how much capacity each product will need. Instead it starts from a **seed matrix**: a table of every **eligible SKU × availability zone** combination, each initialised with a reserved count of **0**. These zero-count entries are called **seed reservations** — governed placeholders that say “this SKU is allowed in this zone” without reserving anything yet.

- A seed reservation only grows above 0 when the **product team's budget governance** approves it. Capacity costs money, so the product team owns the decision to fund it. This extends the existing seed model (CAP-009).
- Because the matrix begins at 0, there is no speculative spend — capacity is reserved only when a real, funded need appears.

A `SeedMatrixEntry` records the SKU, zone, environment, current count, and governance status. A reservation is tagged `SEED` (still at/for the placeholder) or `ACTIVE` (funded and holding capacity).

5.11.3 One zonal group per environment (CAP-023)

Earlier sections described three CRGs per region (Prod, NonProd, DR). v2.5 states how those groups are built inside Azure.

Each environment gets **one zonal Capacity Reservation Group**. The region's zones are set when the group is created and cannot be added later. Inside the group, ACRME keeps **one reservation per VM size per zone**. Capacity reserved in zone 1 is not available in zone 2. A separate group per zone is allowed when a smaller blast radius is wanted; Azure does not require it, because the reservation itself is already per zone.

A group created **without zones** is not a region-wide pool. Azure picks one zone when the first reservation is created, and every VM in that group has to be deployed without a zone. Zonal and non-zonal reservations cannot sit in the same group. The `reg` name is only for a SKU that cannot take a zonal reservation, and the pinned zone is recorded.

Names still come from the naming convention (OPS-006). The default name is `crg-<env>-<region>-zonal`.

5.11.4 Reactive discovery — filling gaps automatically (CAP-024)

When a request arrives for a SKU/zone combination that **isn't yet backed by a CRG**, ACRME does not simply fail. It **reactively discovers** the gap and **auto-creates** the missing reservation against the seed matrix, then **raises a scope-file governance item** so the addition is reviewed rather than silently accepted. This reconciles the earlier reactive-discovery governance requirement (CAP-019). Each auto-created reservation is stamped with its provenance (reactive vs. seeded vs. planned), and it still respects the eligibility rules in 5.11.1.

5.11.5 Spreading VMs evenly across zones (PLC-011)

Within the region already chosen, ACRME aims to spread a workload's VMs **evenly across the zones** — about $1 / \text{zone_count}$ per zone ($\approx 33\%$ each in a three-zone region). This is a placement **target**, not just the ϵ “zone diversity” scoring nudge used during region selection:

- When placing a new VM, the engine **prefers the most under-represented zone** (the one with the fewest VMs so far).
- When a new deployment or a growth event would tilt the balance **beyond a configured tolerance** (C-13), the engine raises a **rebalancing recommendation/action** to move things back toward even — subject to approval and whether Azure can actually accommodate the move.

The skew maths is simple:

Even Zone Share	= $1 / \text{Zone Count}$	($\approx 33\%$ in a 3-zone region)
Target VMs per Zone	= $\text{round}(\text{Workload VM Count} / \text{Zone Count})$	
Zone Skew(z)	= $\text{VMs in zone } z - \text{Target VMs per Zone}$	
Max Skew	= $\text{largest } \text{Zone Skew}(z) \text{ across zones}$	
Rebalance Trigger	= $\text{Max Skew} > \text{Configured Skew Tolerance (C-13)}$	
Preferred Placement Zone	= the zone with the fewest VMs (argmin)	

Even distribution bounds how exposed a workload is if a single zone fails, and it keeps the per-zone reservations (5.11.3) balanced. (Formal version: Appendix A.9.)

5.11.6 Predictable names for everything (OPS-006, C-12)

Finally, v2.4 pins down a **deterministic naming convention** for resource groups, CRGs, and subscriptions, backed by a **counter (C-12)** that guarantees uniqueness. v2.5 names the default group `crg-<env>-<region>-zonal`. `reg` is only the no-zone exception. In plain terms: given the same inputs, ACRME always produces the **same, collision-free name** — so resources are predictable, auditable, and RBAC scopes attach to stable identities instead of ad-hoc names.

6. Scoring Weight Invariant

6.1 The Rule

The five placement scoring weights must always sum to exactly 1.0:

$$\alpha + \beta + \gamma + \delta + \varepsilon = 1.0$$

$$0.30 + 0.20 + 0.25 + 0.15 + 0.10 = 1.00 \checkmark$$

This is not a convention. It is a mathematical requirement for the scores to be meaningful.

6.2 Why 1.0 Is Required

Every component is clamped to [0, 1]. When weights sum to exactly 1.0:

- The **best possible region** (all components = 1.0) scores exactly **1.0**.
- The **worst possible region** (all components = 0.0) scores exactly **0.0**.
- Every real region scores **somewhere between 0 and 1**.
- Scores from different regions are **directly comparable**: 0.75 is always better than 0.60, in every run, under every policy version.

6.3 What Breaks When the Weights Don't Sum to 1.0

If weights sum to more than 1.0 (e.g., 1.10):

The best possible score inflates above 1.0. A region with all perfect components scores 1.10 instead of 1.0. Any downstream threshold calibrated against a [0, 1] range — for example, “do not place in any region scoring below 0.60” — is now calibrated incorrectly. A mediocre region may clear the threshold simply because scores are inflated.

If weights sum to less than 1.0 (e.g., 0.90):

All scores are uniformly compressed into [0, 0.90]. A region with all perfect components scores 0.90 instead of 1.0. A region that should score 0.67 now scores 0.60. Threshold gates calibrated against [0, 1] now behave incorrectly in the other direction — some deployments are incorrectly blocked.

Cross-policy comparisons also break: if one version had weights summing to 1.0 and the next has them

summing to 0.90, a score of 0.65 means something different under each version. Historical score trend data becomes uninterpretable.

6.4 Tuning the Weights — The Pie Rule

Think of the five weights as slices of a pie. The total pie is always exactly 100%. If one slice grows, the others must shrink by the same total amount.

Example — increasing headroom importance:

Before: $\alpha=0.30, \beta=0.20, \gamma=0.25, \delta=0.15, \varepsilon=0.10 \rightarrow \text{sum} = 1.00 \checkmark$
 After: $\alpha=0.40, \beta=0.15, \gamma=0.25, \delta=0.10, \varepsilon=0.10 \rightarrow \text{sum} = 1.00 \checkmark$ (took 0.05 from β , 0.05 from δ)
 Wrong: $\alpha=0.40, \beta=0.20, \gamma=0.25, \delta=0.15, \varepsilon=0.10 \rightarrow \text{sum} = 1.10 \times$ (forgot to reduce)

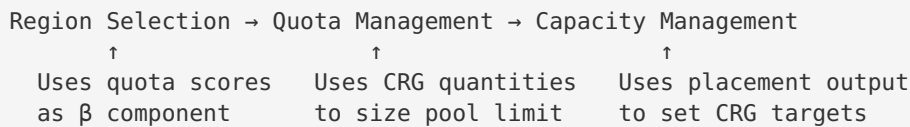
6.5 Validation Requirement — The Policy Load Gate

The Config/Scope-File Service **must** validate the weight sum every time a new PlacementPolicy version is loaded. This validation must occur **before** the policy version is published to the placement engine. A policy that fails this check must be rejected with a recorded configuration error; the placement engine must never receive a policy with an invalid weight sum.

See Section 6 of the Calculation Logic Reference and Section 8.2 of the Technical Design Document for the formal validation rule specification.

7. How the Three Domains Connect

The three domains are not independent. They feed into each other:



Flow for a new customer deployment:

1. **Region Selection** scores all candidate regions and picks the best Prod anchor. The quota score (β component) uses current quota headroom from Domain 2. The capacity/headroom score (α component) uses current CRG free slots from Domain 3.
2. **Quota Management** checks that the subscription in the selected region has enough quota limit to accept the new deployment (Scenario 19, gate 7). If not, the deployment is blocked with `QUOTA_DEFICIT` until a quota increase request to Microsoft is approved.
3. **Capacity Management** checks that a Capacity Reservation exists and has enough quantity for the deployment (Scenario 19, gate 6). If not, the deployment is blocked with `RESERVATION_DEFICIT` until the CRG is expanded.

Only when all three domains return green does the readiness gate emit `READY` and allow the deployment to proceed.

Flow for a DR event:

1. The engine transitions to `DR_EVENT_ACTIVE`.
2. The DR Orchestrator consults the SourceDestinationDRIndex to identify which standby VMs need to be activated and where.
3. **Capacity Management** begins staged activation (Scenario 18): bootstrap headroom first, then CRG expansion if needed, then CVAL release, then quota draw.
4. **Quota Management** confirms the shared pool has headroom for the DR draw (single-pool model: no cross-group transfer needed — the DR earmark is already inside the pool).
5. **Region Selection** is not re-run — the Customer Seed Record is authoritative. Failback restores the original seed without re-derivation.

8. Quick Reference — Key Numbers

8.1 Configurable Parameters Reference

ACRME is designed to be **configuration-driven**. Most key values are **policy defaults stored in `PlacementPolicy`**, not hardcoded constants. This table lists all tunable parameters, their baseline defaults, and how they're configured.

Region Selection — Scoring Weights

Parameter	Baseline Default	Configurable?	Scope	Validation
alpha (α) — Capacity headroom	0.30	✓ Yes	Per product/region/env/SKU	Must sum to 1.0 (± 0.001)
beta (β) — Quota headroom	0.20	✓ Yes	Per product/region/env/SKU	Must sum to 1.0 (± 0.001)
gamma (γ) — Distribution fairness	0.25	✓ Yes	Per product/region/env/SKU	Must sum to 1.0 (± 0.001)
delta (δ) — DR readiness	0.15	✓ Yes	Per product/region/env/SKU	Must sum to 1.0 (± 0.001)
epsilon (ϵ) — Zone diversity	0.10	✓ Yes	Per product/region/env/SKU	Must sum to 1.0 (± 0.001)
Weight sum	1.00	✗ No (invariant)	—	Enforced at policy load
Weight sum tolerance	0.001	✗ No (fixed)	—	IEEE 754 rounding allowance

How to change: Update `PlacementPolicy` config file → Config/Scope-File Service validates at load time → if valid, policy is published. See Section 3.8 for tuning guidance.

Quota Management — Growth Buffers & Headroom Floors

Parameter	Baseline Default	Configurable?	Scope	What it controls
prod_growth_buffer	0.20 (20%)	✓ Yes	Per product/region/SKU	Quota headroom above current Prod usage
non-prod_growth_buffer	0.20 (20%)	✓ Yes	Per product/region/SKU	Quota headroom above current NonProd usage
emergency_transfer_pct	0.30 (30%)	✓ Yes	Per product/region	% of potential DR demand held as emergency transfer headroom
min_prod_quota_headroom_vcpu	20 vCPU	✓ Yes	Per product/region/SKU	Minimum absolute quota headroom floor for Prod
min_nonprod_quota_headroom_vcpu	20 vCPU	✓ Yes	Per product/region/SKU	Minimum absolute quota headroom floor for NonProd
min_dr_headroom_vcpu	16 vCPU	✓ Yes	Per product/region/SKU	Minimum absolute quota headroom floor for DR

How to change: Update `PlacementPolicy` config file → no validation gate required (arbitrary values permitted). Changes take effect on next policy load.

Capacity Management — Auto-Increase Thresholds & Reconciliation

Parameter	Baseline Default	Configurable?	Scope	What it controls
prod_autoincrease_threshold	0.20 (20%)	✓ Yes	Per product/region/SKU	Trigger capacity increase when Prod headroom drops below 20% (80% utilized)
non-prod_autoincrease_threshold	0.20 (20%)	✓ Yes	Per product/region/SKU	Trigger capacity increase when NonProd headroom drops below 20%
dr_autoincrease_threshold	0.35 (35%)	✓ Yes	Per product/region/SKU	Trigger capacity increase when DR headroom drops below 35% (65% utilized)
autoincrease_debounce_cooldown	30 min	✓ Yes	Per region + CRG type	Minimum time between auto-increase triggers for same region + CRG
reconciliation_interval_seconds	360 sec (6 min)	✓ Yes	Global	How often the engine checks allocated vs reserved capacity
configured_buffer (Prod)	TBD (e.g., 1 slot)	✓ Yes	Per product/region/env/SKU	Reconciliation floor buffer above allocated VMs
configured_buffer (NonProd)	TBD	✓ Yes	Per product/region/env/SKU	Reconciliation floor buffer above allocated VMs
configured_buffer (DR)	TBD	✓ Yes	Per product/region/env/SKU	Reconciliation floor buffer above allocated VMs

How to change: Update `PlacementPolicy` config file → no validation gate required. Changes take effect on next policy load. See Section 5.4 for auto-increase threshold tuning guidance.

Capacity Management — Forecast & Alerts

Parameter	Baseline Default	Configurable?	Scope	What it controls
<code>fore-cast_horizon</code>	{30, 60, 90} days	✓ Yes	Per forecast request	Prediction window for peak VM demand
<code>fore-cast_quota_alert_threshold</code>	0.80 (80%)	✓ Yes	Global	Quota utilization % that triggers ForecastApproachingQuotaLimit alert
<code>fore-cast_quota_alert_lead_time</code>	14 days	✓ Yes	Global	Minimum lead time for quota alert before predicted breach

How to change: Update `PlacementPolicy` config file or forecast service config → changes take effect on next policy load or forecast run.

DR Management — Bootstrap Targets

Parameter	Baseline Default	Configurable?	Scope	What it controls
<code>dr_bootstrap_target</code> (per workload)	TBD (lean, ~5–20%)	✓ Yes	Per workload/product/region/zone/VM family/subscription	Minimum DR capacity to pre-stage (ENV-005)
<code>dr_coverage_target</code> (scoring only)	TBD (varies per customer SLA)	✓ Yes	Per customer/product	Normalization target for the scoring 6 component only — NOT a DR sizing ratio (sizing uses max-not-sum, A.6/DR-017; fixed <code>dr_ratio_*</code> retired)

How to change: Update `PlacementPolicy` config file → no validation gate required. DR bootstrap cannot be implicitly zero (ENV-006) — zero requires explicit approved policy exception.

Fixed / System Constants (Not Configurable)

Parameter	Value	Why it's fixed
Weight sum invariant	1.00	Mathematical requirement for score normalization
Weight sum tolerance	0.001	IEEE 754 floating-point precision allowance
Component clamp range	[0, 1]	Normalization requirement for weighted scoring
Placement score range	[0.0, 1.0]	Direct consequence of weight sum = 1.0 and clamp [0, 1]
Debounce cooldown minimum	30 min	Operational safety to prevent request thrashing

Configuration Change Process

All configurable parameters follow the same update process:

1. **Update the `PlacementPolicy` configuration file** (JSON/YAML, stored in version control)
2. **Config/Scope-File Service validates** (for weight-sum gate only; other params have no validation)
3. **If valid, policy version is published** to the Placement Engine
4. **Engine records `policy_version`** in every placement decision (audit trail)
5. **Changes take effect** on the next placement/reconciliation/forecast cycle

No code deployment is required to change any configurable parameter. Only a policy configuration file update and service restart/reload.

Monitoring after change:

- Track the metrics in Section 3.8.6 (weight changes) or Section 5.4.1 (threshold changes)
- Compare placement/capacity decisions before and after the change
- Monitor auto-increase trigger frequency, capacity utilization, quota request rates

Rollback: Revert the `PlacementPolicy` config file to the previous version and republish.

8.2 Quick Reference — All Key Numbers

This table combines all key values (configurable and fixed) for quick lookup:

Parameter	Value	Domain
α (headroom/capacity weight)	0.30	Region Selection
β (quota headroom weight)	0.20	Region Selection
γ (distribution fairness weight)	0.25	Region Selection
δ (DR readiness / overflow weight)	0.15	Region Selection
ϵ (zone diversity weight)	0.10	Region Selection
Weight sum (must equal)	1.00	Region Selection
Weight sum validation tolerance	0.001	Region Selection
Prod growth buffer	20%	Quota Management
NonProd growth buffer	20%	Quota Management
Emergency transfer headroom	30% of potential DR demand	Quota Management
Min Prod quota headroom	20 vCPU	Quota Management
Min NonProd quota headroom	20 vCPU	Quota Management
Min DR headroom	16 vCPU	Quota Management
Prod auto-increase threshold	20% headroom remaining	Capacity Management
NonProd auto-increase threshold	20% headroom remaining	Capacity Management
DR auto-increase threshold	35% headroom remaining	Capacity Management
Auto-increase debounce cooldown	30 min per region + CRG type	Capacity Management
Forecast horizons	30 / 60 / 90 days	Capacity Management
Quota alert lead time	14 days at 80% of limit	Capacity Management
Reconciliation loop interval	6 minutes (configurable)	Capacity Management
US Standard Capacity Regions (3-region)	West US 3 · Central US · Canada Central	All domains

Parameter	Value	Domain
EU Standard Capacity Regions (2-region)	Switzerland North · Sweden Central	All domains
Australia Standard Capacity Regions (2-region)	Australia East · Australia Southeast	All domains
Asia Pacific Standard Capacity Regions (2-region)	East Asia · Southeast Asia	All domains
Middle East Standard Capacity Regions (2-region)	UAE North · Saudi Arabia Central	All domains
Japan East (Asia Pacific)	Pending business confirmation — not selectable until added to config	Region Selection
Middle East DR status	[Amended v2.4] Cross-geo DR into a weighted-selected Europe Standard region (PLC-010b, DR-020, DEC-001 RESOLVED)	Region Selection
CVAL + DR co-location	Mandatory (PLC-010a) in EU/ Australia/Asia Pacific 2-region geographies. [Amended v2.4] In the Middle East , CVAL co-locates with Prod locally and DR is cross-geo in Europe (PLC-010b)	Region Selection
Region catalogue	Configurable & versioned (REG-001) — geographies, regions, class, distribution model	All domains

Document version 1.1 — reconciled to Requirements Baseline v2.4 (7 September 2026); originally 2 September 2026.

Source: ACRME Q&A design walkthrough session, Calculation Logic Reference, Requirements Baseline v2.4.

Next review: when Calculation Logic Reference is next revised or when POC-001/POC-011 results are available.